Analyse Dynamique d'Application Android

Laboratoire MobSF - DIVA Application

Cours : Sécurité des applications mobiles

Analyste : Omar Haouani

APK cible : DivaApplication.apk

Date : 6 avril 2026

Table des matières

1 Introduction **3**
1.1 Objectifs du Laboratoire 3

2 Environnement Technique **3**

3 Nature de l'Émulateur Utilisé **3**

4 Phase 1 : Clonage de MobSF **5**

5 Phase 2 : Lancement de MobSF via Docker **5**
5.1 Commande d'exécution Docker 5
5.2 Informations de démarrage 5

6 Phase 3 : Accès à l'Interface Web **6**
6.1 Connexion à MobSF 6
6.2 Page d'accueil 7

7 Phase 4 : Analyse Statique de l'APK **8**
7.1 Informations Générales 8
7.2 Tableau de Bord d'Analyse 8

8 Phase 5 : Analyse des Vulnérabilités **8**
8.1 Vulnérabilités Identifiées 8
8.2 Corrélation OWASP MASVS 9

9 Phase 6 : Analyse Dynamique **9**
9.1 Démarrage de l'Analyse Dynamique 9
9.2 Fonctionnalités de MobSF en Dynamique 9
9.3 Exemples de Tests avec DIVA 9

10 Recommandations **9**
10.1 Priorité Haute - Corriger Immédiatement 9
10.2 Priorité Moyenne - Planifier 10
10.3 Priorité Basse - Amélioration Continue 10

11 Liste de Contrôle Finale **10**

12 Conclusion **10**

13 Annexes **10**

13.1 Ressources Documentaires 10

13.2 Fichiers Produits 11

1 Introduction

Ce laboratoire porte sur l'**analyse dynamique** d'une application Android à l'aide de **MobSF (Mobile Security Framework)**. L'application cible est `DivaApplication.apk` (Damn Insecure and Vulnerable Application), une application volontairement vulnérable conçue pour l'apprentissage de la sécurité mobile.

1.1 Objectifs du Laboratoire

- Comprendre l'analyse dynamique (runtime) d'une application Android
- Configurer un émulateur propre sans services Google
- Installer et lancer MobSF via Docker
- Tester l'APK vulnérable DIVA en environnement dynamique
- Détecter des vulnérabilités en temps réel (logs, trafic, stockage, intents)

2 Environnement Technique

Outil	**Version / Spécification**
MobSF	v4.5.0
Docker	Desktop pour Windows
AVD	Pixel 5 API 37 (sans Play Store)
ADB	Platform-Tools
APK cible	DivaApplication.apk (1.43 Mo)

3 Nature de l'Émulateur Utilisé

L'émulateur utilisé est un **AVD sans Play Store**. Ce choix est délibéré pour l'analyse de sécurité car :

- **Absence de bruit de fond** : Pas de services Google Play, Firebase ou autres processus en arrière-plan
- **Trafic pur** : Les logs et le trafic réseau ne contiennent que l'application testée
- **Compatibilité MobSF** : L'émulateur peut être rooté et configuré pour l'analyse dynamique
- **Performances optimales** : Architecture x86_64 sur PC

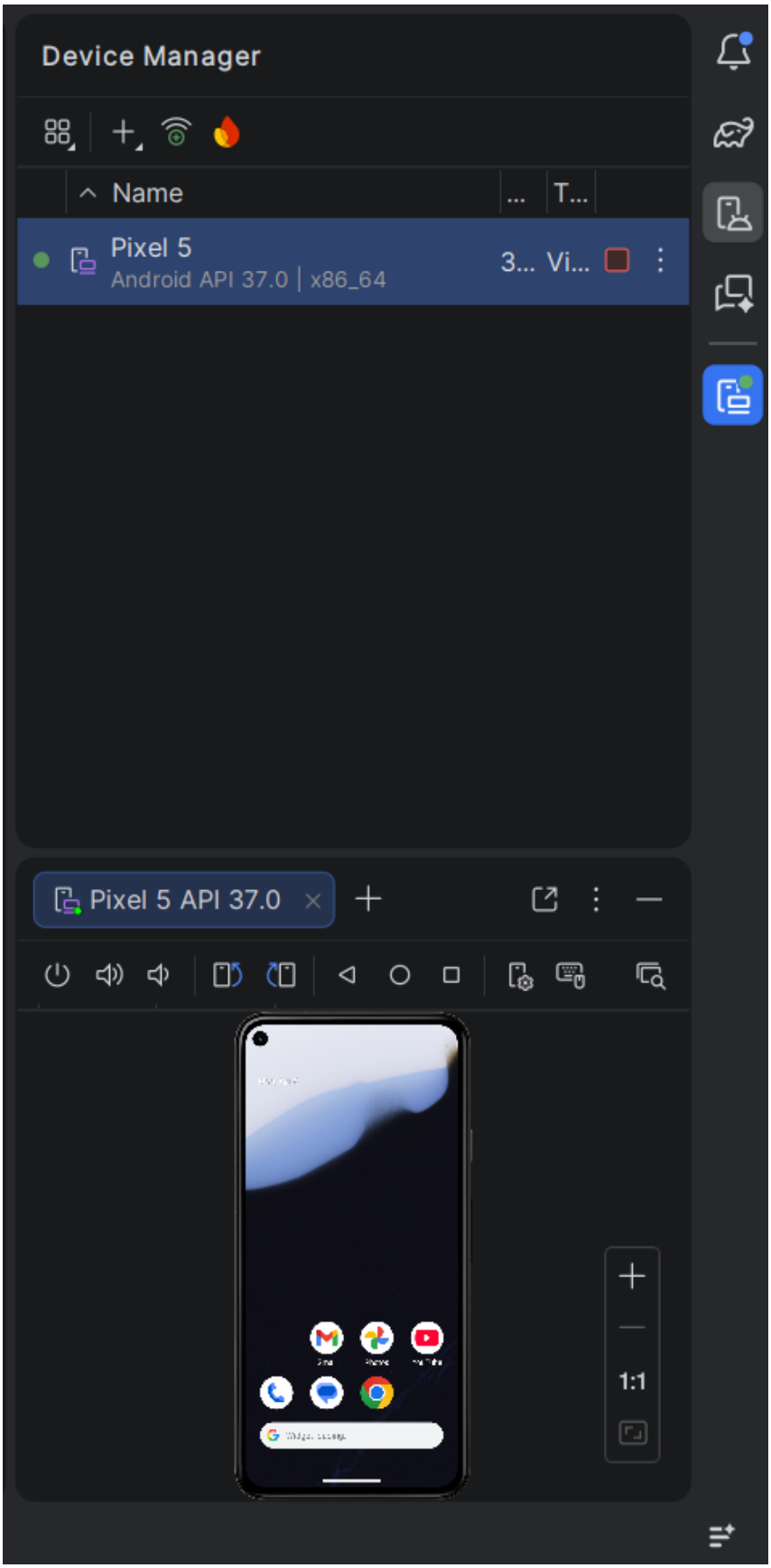

FIGURE 1 – Device Manager - Pixel 5 API 37 sans Play Store

4 Phase 1 : Clonage de MobSF

```
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.
    git
cd Mobile-Security-Framework-MobSF
```

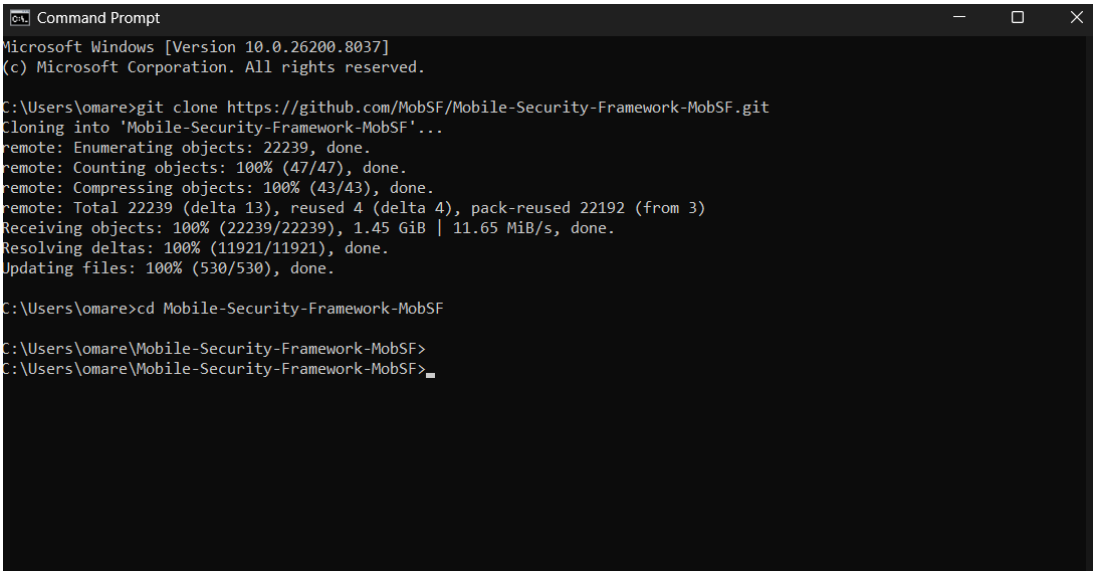

FIGURE 2 – Clonage du dépôt MobSF

5 Phase 2 : Lancement de MobSF via Docker

5.1 Commande d'exécution Docker

```
docker run -it --rm -p 8000:8000 -e MOBSF_ANALYZER_IDENTIFIER=
    emulator-5554 opensecurity/mobile-security-framework-mobsf:
    latest
```

5.2 Informations de démarrage

Paramètre	Valeur
Version MobSF	4.5.0
REST API Key	48f82d5d308a460e4e10d44cdc6977b0ea418b05559ac0e31a4327990adfac89
Identifiants par défaut	mobsf / mobsf
OS Environment	Debian 12 (WSL2)

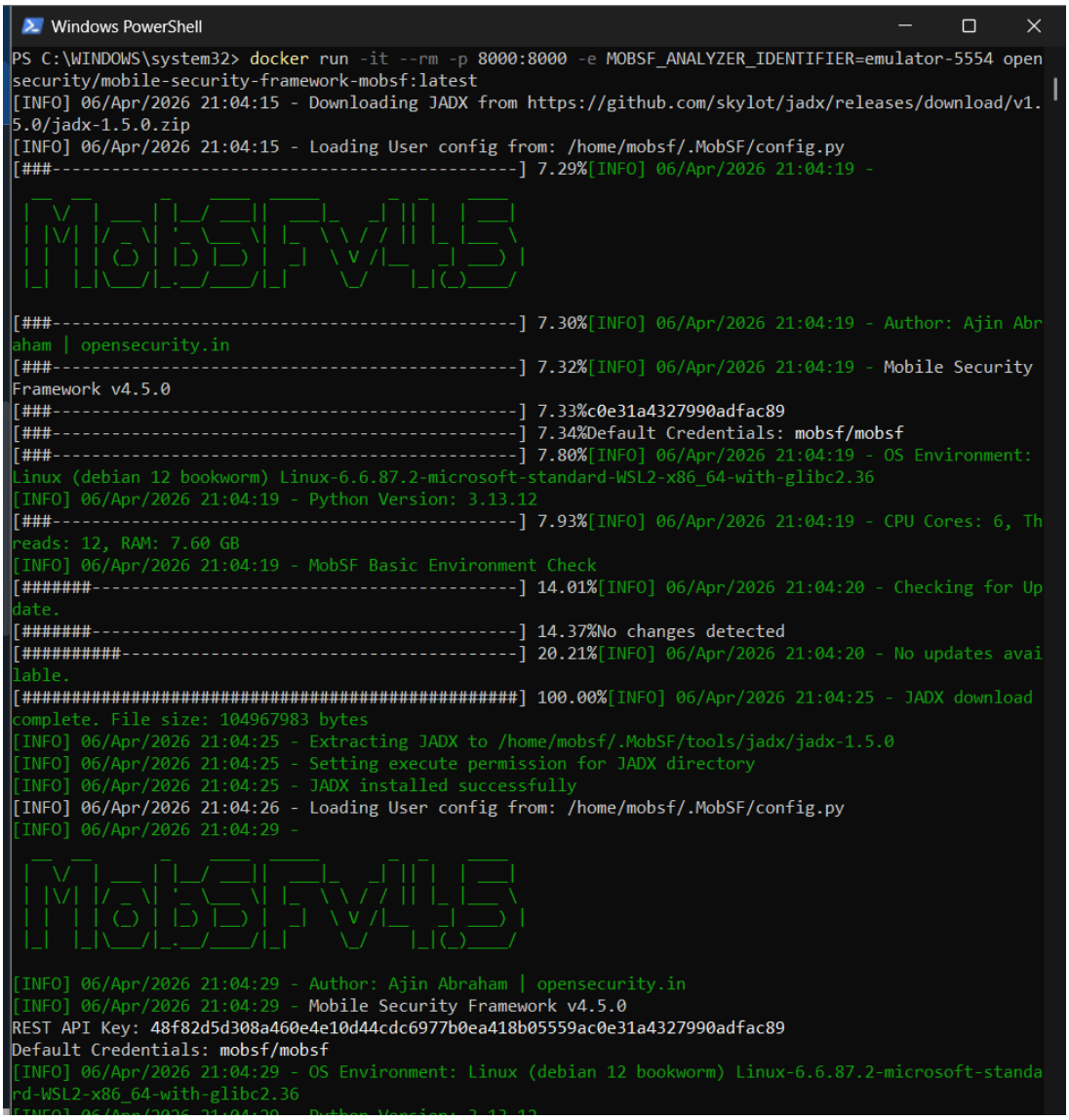

FIGURE 3 – Démarrage de MobSF dans Docker

6 Phase 3 : Accès à l'Interface Web

6.1 Connexion à MobSF

Ouvrir un navigateur à l'adresse : `http://127.0.0.1:8000`

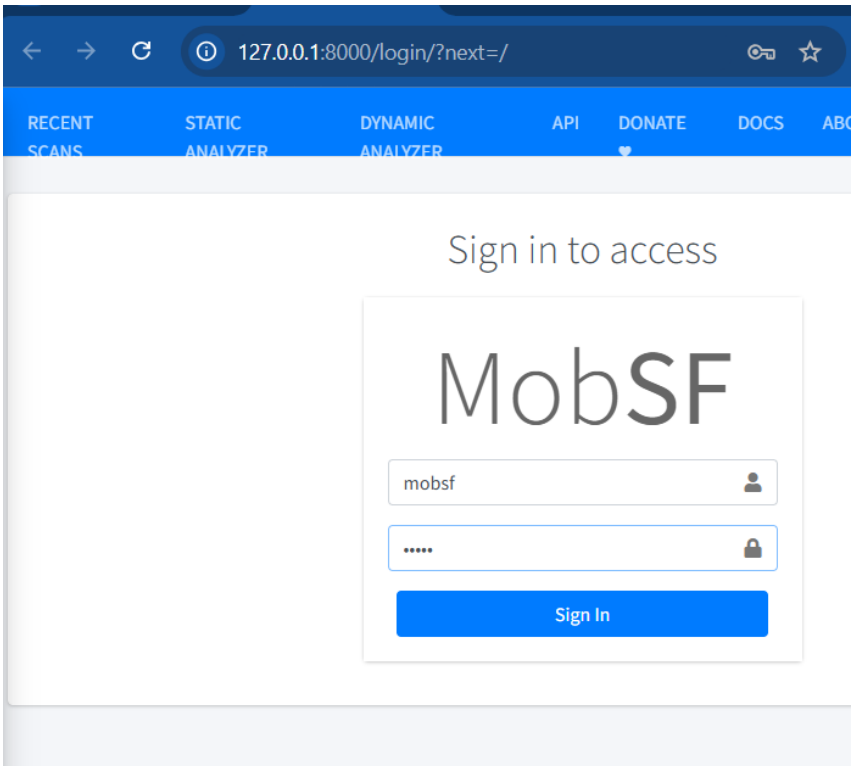

FIGURE 4 – Interface de connexion MobSF

6.2 Page d'accueil

Après connexion, la page principale permet l'upload d'APK.

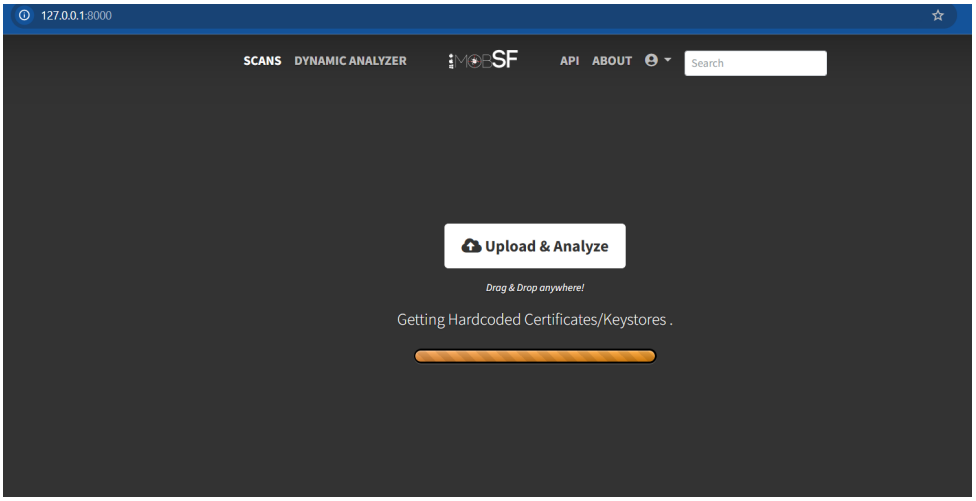

FIGURE 5 – Page d'accueil MobSF - Upload & Analyze

7 Phase 4 : Analyse Statique de l'APK

7.1 Informations Générales

Champ	Valeur
Nom du fichier	DivaApplication.apk
Taille	1.43 Mo
MD5	82ab8b2193b3cfb1c737e3a786be363a
SHA-256	5cefc51fce9bd760b92ab2340477f4dda84b4ae0c5d04a8c9493e4fe34fab7c5
Nom de l'application	Diva
Package	jakhar.aseem.diva
Activité principale	jakhar.aseem.diva.MainActivity
Version	1.0

7.2 Tableau de Bord d'Analyse

L'interface MobSF affiche les scores de sécurité et les options d'analyse.

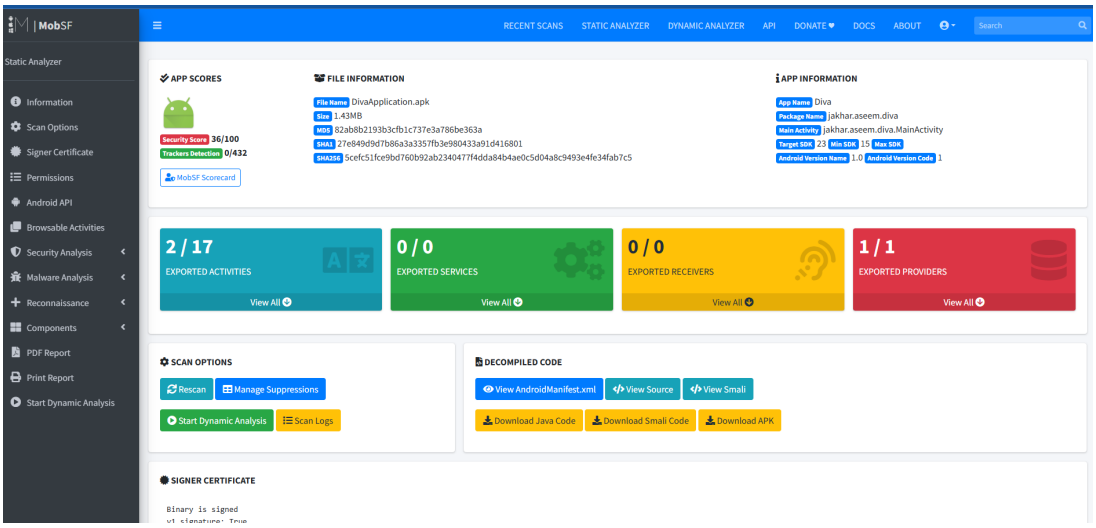

FIGURE 6 – Tableau de bord de l'analyse statique

8 Phase 5 : Analyse des Vulnérabilités

8.1 Vulnérabilités Identifiées

Vulnérabilité	Sévérité	Description
minSdkVersion=15	Élevée	Support d'Android 4.0.3 (2011) - appareils non corrigés
targetSdkVersion=23	Moyenne	SDK cible Android 6.0 (2015) - versions dépassées
allowBackup non défini	Moyenne	Par défaut true - risque d'extraction des données
Logging insecure	Élevée	Présence de logs sensibles dans le code

Hardcoded credentials	Élevée	Identifiants codés en dur dans le code source
Composants exportés	Moyenne	Activities/Receivers accessibles depuis d'autres apps

8.2 Corrélation OWASP MASVS

Découverte	Référence MASVS	Statut
minSdkVersion=15	MASVS-CODE-1	Non conforme
targetSdkVersion=23	MASVS-CODE-1	Non conforme
Logs sensibles	MASVS-STORAGE-6	Non conforme
Hardcoded secrets	MASVS-STORAGE-2	Non conforme
Composants exportés	MASVS-CODE-3	À vérifier

9 Phase 6 : Analyse Dynamique

9.1 Démarrage de l'Analyse Dynamique

L'analyse dynamique permet d'observer l'application en cours d'exécution.

9.2 Fonctionnalités de MobSF en Dynamique

- **Runtime Logs** : Capture des logs en temps réel
- **Network Traffic** : Interception du trafic HTTP/HTTPS
- **Frida Instrumentation** : Hooking de méthodes Java
- **File Monitor** : Surveillance des fichiers créés
- **Intent Monitor** : Capture des intents non sécurisés

9.3 Exemples de Tests avec DIVA

Challenge	Comportement observé	Vulnérabilité
Insecure Logging	Données sensibles dans logcat	Exposition d'informations
Hardcoded Credentials	Identifiants visibles dans le code	Secret exposé
Insecure Storage	Fichiers écrits en clair	Données non protégées
Access Control	Intents non sécurisés	Contournement d'authentification

10 Recommandations

10.1 Priorité Haute - Corriger Immédiatement

- Supprimer tous les logs sensibles du code de production
- Ne jamais stocker d'identifiants en clair dans le code
- Désactiver `android:allowBackup="false"`

10.2 Priorité Moyenne - Planifier

— Augmenter `minSdkVersion` à 21 minimum
— Augmenter `targetSdkVersion` à 33 (Android 13)
— Protéger tous les composants exportés

10.3 Priorité Basse - Amélioration Continue

— Implémenter l'obfuscation de code (ProGuard/R8)
— Mettre en place des tests de sécurité automatisés
— Former les développeurs aux bonnes pratiques

11 Liste de Contrôle Finale

x Création de l'AVD sans Play Store
x Clonage du dépôt MobSF
x Lancement de MobSF via Docker
x Connexion à l'interface web
x Upload et analyse statique de DIVA
x Identification des vulnérabilités
x Démarrage de l'analyse dynamique
x Tests des challenges DIVA
x Documentation des découvertes
x Rédaction des recommandations

12 Conclusion

Ce laboratoire a démontré l'efficacité de MobSF pour l'analyse dynamique d'applications Android. L'application DIVA, volontairement vulnérable, a permis d'identifier plusieurs failles classiques :

1. Stockage insecure de données sensibles
2. Logs exposant des informations confidentielles
3. Identifiants codés en dur
4. Composants Android mal configurés

L'utilisation d'un AVD sans Play Store combiné à MobSF via Docker constitue un environnement d'analyse professionnel, reproductible et isolé.

13 Annexes

13.1 Ressources Documentaires

— MobSF sur GitHub

— OWASP MASVS/MASTG
— DIVA Android - Projet Payatu
— Documentation Docker

13.2 Fichiers Produits

— Rapport d'analyse statique MobSF (PDF)
— Logs d'exécution Docker
— Captures d'écran de l'analyse dynamique

Document réalisé dans le cadre du cours de Sécurité des applications mobiles
Omar Haouani - 6 avril 2026